

THIS KEYWORD

THIS KEYWORD

This keyword – Represent instance variables of current object

Class ThisDemo

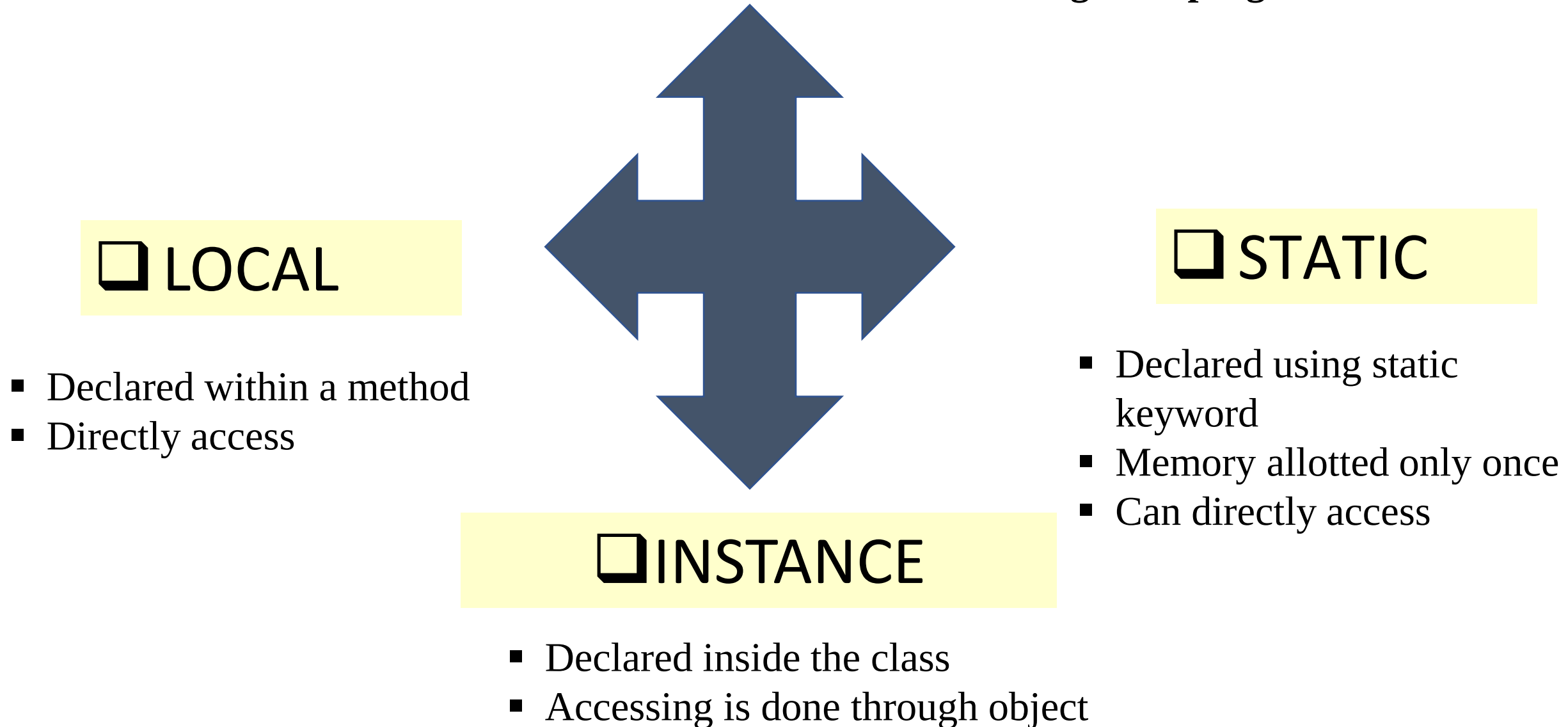
```
{
    int a = 10; //instance variable
    void display()
    {
        System.out.println(a); // compiler will
        treat as this.a – this represent variable of current class or
        current object
    }
    public static void main(String arg[])
    {
        ThisDemo obj = new ThisDemo();
        obj.display();
    }
}
```

- We know that every class have variables and methods. There are three types of variables – local, instance static . Instance variable – declared inside class outside method. So in order to access instance variable we need to create object.
- In order to access instance variable we are using this keyword
- Explicitly we no need to write this.a in every case. But there is some case we need to use this keyword



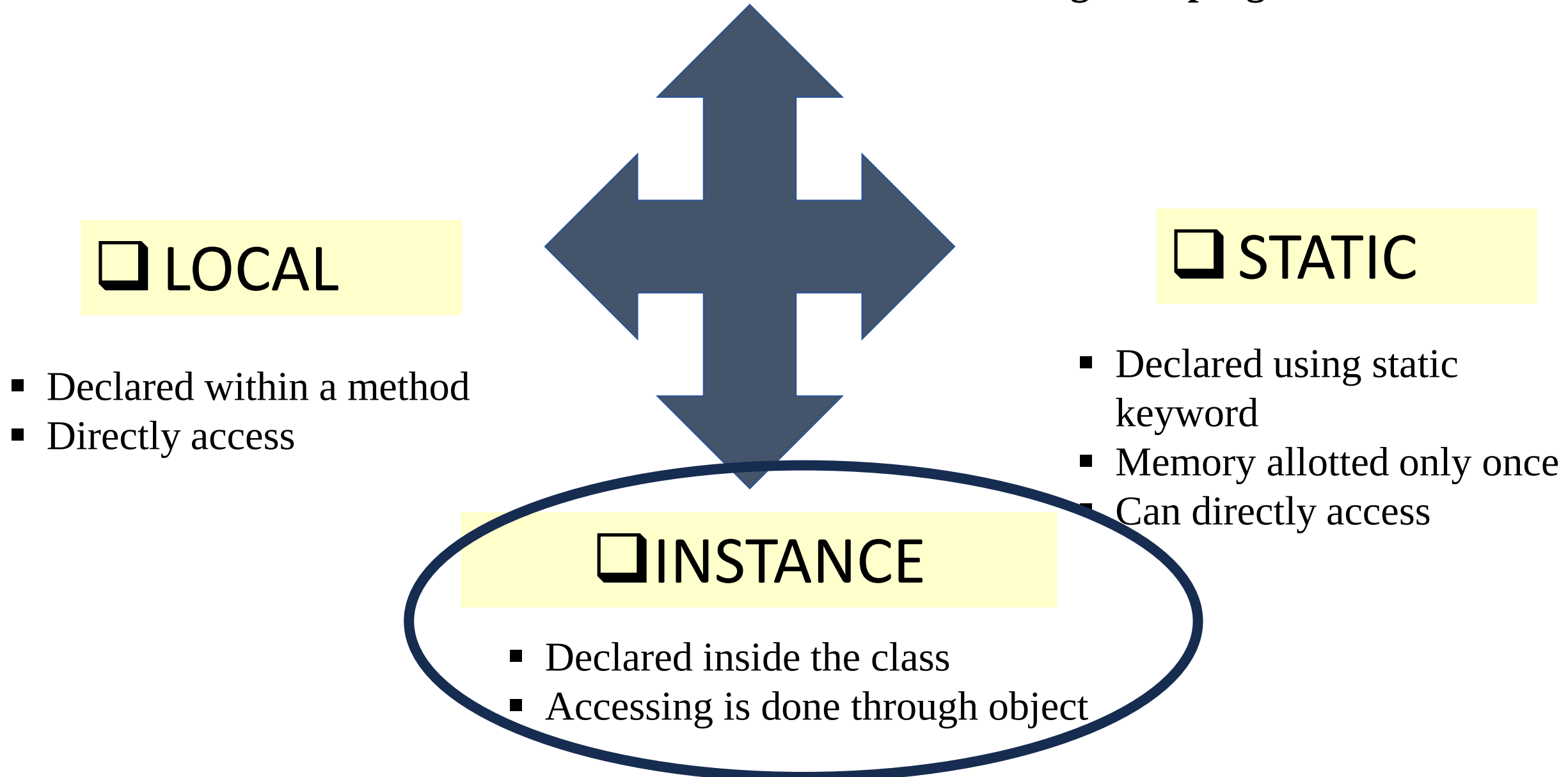
VARIABLES

Variable in Java is a **data container** that saves the data values during Java program execution.



VARIABLES

Variable in Java is a **data container** that saves the data values during Java program execution.



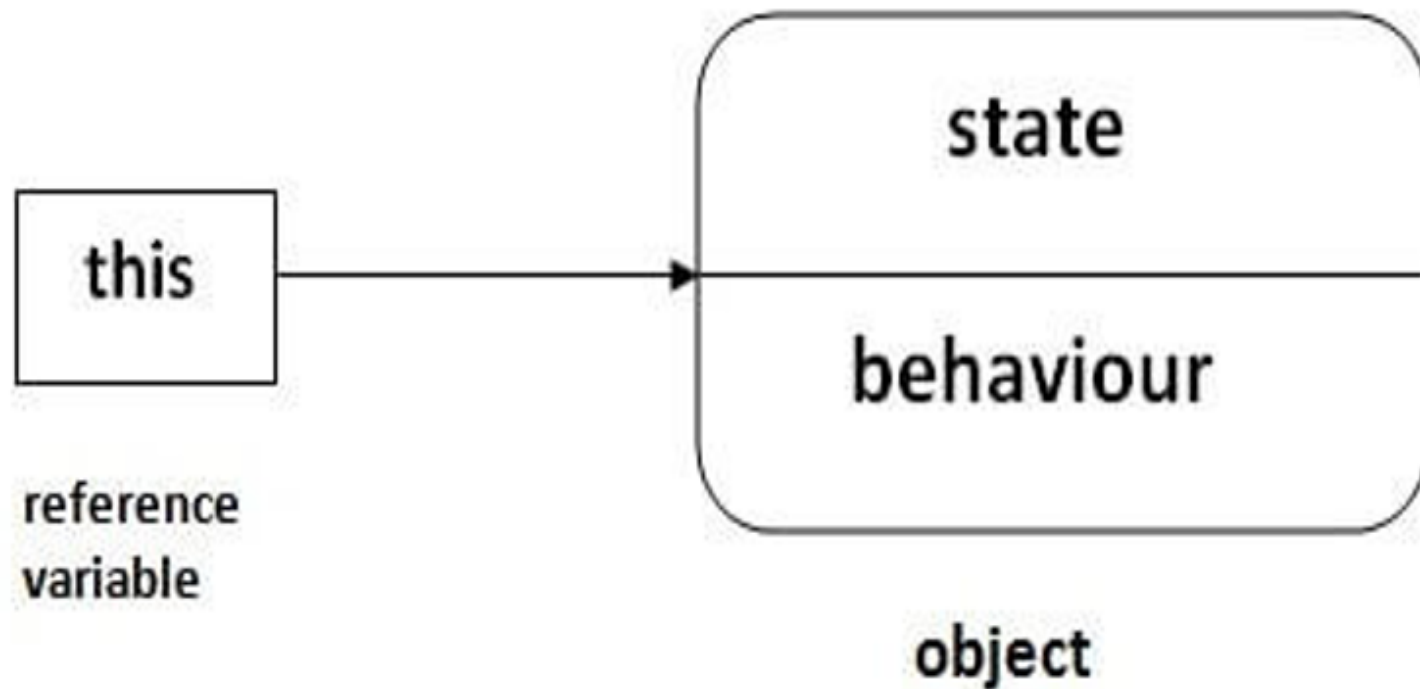
THIS KEYWORD

This keyword – Represent instance variables of current object

```
class ThisDemo {  
    int a = 10; // Instance variable  
  
    void display() {  
        int a = 20; // Local variable  
        System.out.println(a); // Should print the local variable 'a' (20)  
        System.out.println(this.a); // Should print the instance variable 'a' (10)  
    }  
  
    public static void main(String[] args) {  
        ThisDemo obj = new ThisDemo();  
        obj.display();  
    }  
}
```

this keyword in Java

There can be a lot of usage of **Java this keyword**. In Java, this is a **reference variable** that refers to the current object.



Usage of Java this keyword

Here is given the 6 usage of java this keyword.

1. this can be used to refer current class instance variable.
2. this can be used to invoke current class method (implicitly)
3. this() can be used to invoke current class constructor.
4. this can be passed as an argument in the method call.
5. this can be passed as argument in the constructor call.
6. this can be used to return the current class instance from the method.

CONSTRUCTORS IN JAVA

In Java, a constructor is a special type of method that is used to initialize objects. It is called when an instance of a class is created. Constructors have the same name as the class and do not have a return type, not even void. Constructors are used to set the initial state of an object or to perform any setup steps required for the object.

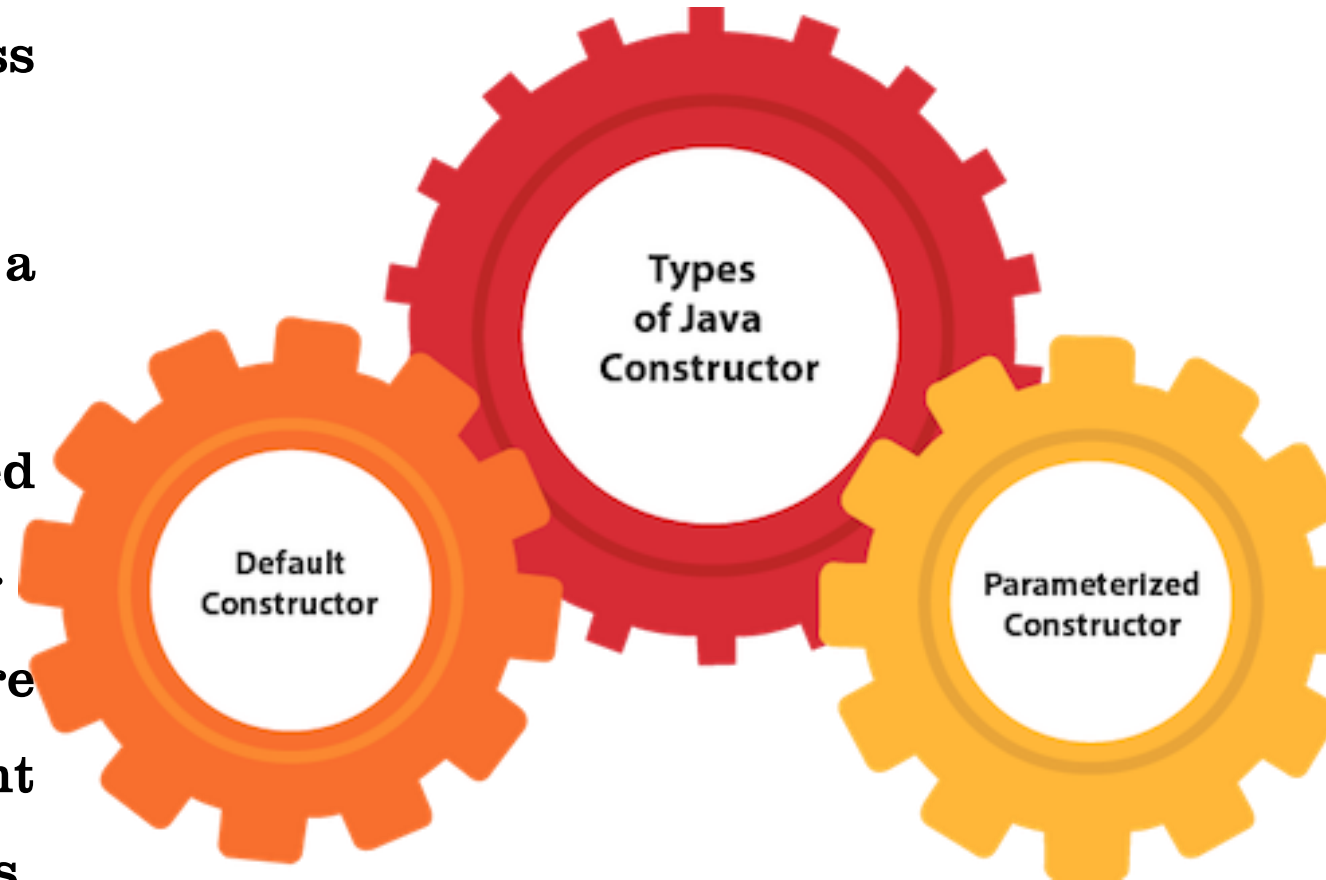
Rules for creating Java constructor

There are two rules defined for the constructor.

- Constructor name must be the same as its class name**
- A Constructor must have no explicit return type**
- A Java constructor cannot be abstract, static, final.**

• Key Points about Constructors:

- ❖ **Same Name as the Class:** Constructors must have the same name as the class within which they are defined.
- ❖ **No Return Type:** They do not specify a return type, not even void.
- ❖ **Automatically Called:** They are called automatically at the time of object creation.
- ❖ **Can Be Overloaded:** A class can have more than one constructor, allowing for different ways of initializing objects of that class. This is known as constructor overloading.



- **1. Default Constructor (No-arg Constructor)**

- **A default constructor is the constructor provided by Java if no other constructors are explicitly defined within a class. It takes no arguments (hence "no-arg") and usually performs simple initializations. If you define any constructor for your class, Java does not provide a default constructor.**

- **Syntax of default constructor:**

- **`<class_name>(){}`**

```
class Employee {  
    String name;  
    int id;  
  
    // Default constructor  
    Employee() {  
        name = "Unknown";  
        id = 0;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Employee employee = new Employee();  
        System.out.println("Name: " + employee.name + ", ID: " + employee.id);  
    }  
}
```

Class with a Default Constructor and Methods

```
class Counter {  
    int count;  
  
    // Default constructor  
    Counter() {  
        count = 0;  
    }  
  
    void increment() {  
        count++;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Counter myCounter = new Counter();  
        myCounter.increment();  
        System.out.println("Count: " + myCounter.count);  
    }  
}
```

PARAMETERIZED CONSTRUCTOR

parameterized constructor in Java is a constructor that includes parameters. This allows you to create and initialize the fields of an object with specific values at the time of object creation, providing a more flexible and controlled way of setting up your object's initial state.

```
class Circle {  
    double radius;  
  
    // Parameterized constructor  
    Circle(double newRadius) {  
        radius = newRadius;  
    }  
    double getArea() {  
        return Math.PI * radius * radius;  
    }  
    public static void main(String[] args) {  
        Circle circle = new Circle(7.0);  
        System.out.println("Area of the circle: " + circle.getArea());  
    }  
}
```

```
class Rectangle {  
    double width;  
    double height;  
    // Parameterized constructor  
    Rectangle(double width, double height) {  
        this.width = width;  
        this.height = height;  
    }  
    double getArea() {  
        return width * height;  
    }  
    public static void main(String[] args) {  
        Rectangle rectangle = new Rectangle(4.0, 5.0);  
        System.out.println("Area of the rectangle: " + rectangle.getArea());  
    }  
}
```


- **Difference between constructor and method in Java**

- **There are many differences between constructors and methods. They are given below.**

Java Constructor	Java Method
A constructor is used to initialize the state of an object.	A method is used to expose the behavior of an object.
A constructor must not have a return type.	A method must have a return type.
The constructor is invoked implicitly.	The method is invoked explicitly.
The Java compiler provides a default constructor if you don't have any constructor in a class.	The method is not provided by the compiler in any case.
The constructor name must be same as the class name.	The method name may or may not be same as the class name.

Constructor Overloading: This occurs when a class has more than one constructor, each with a different parameter list (different number of parameters or types of parameters). Constructor overloading allows creating objects in different ways depending on the information available at the time of instantiation.

Copy Constructor: Java does not provide a built-in copy constructor like some other languages (e.g., C++), but you can define your own. A copy constructor is a parameterized constructor that takes an object of the same class as a parameter and initializes the new object's fields with the same values as the passed object.

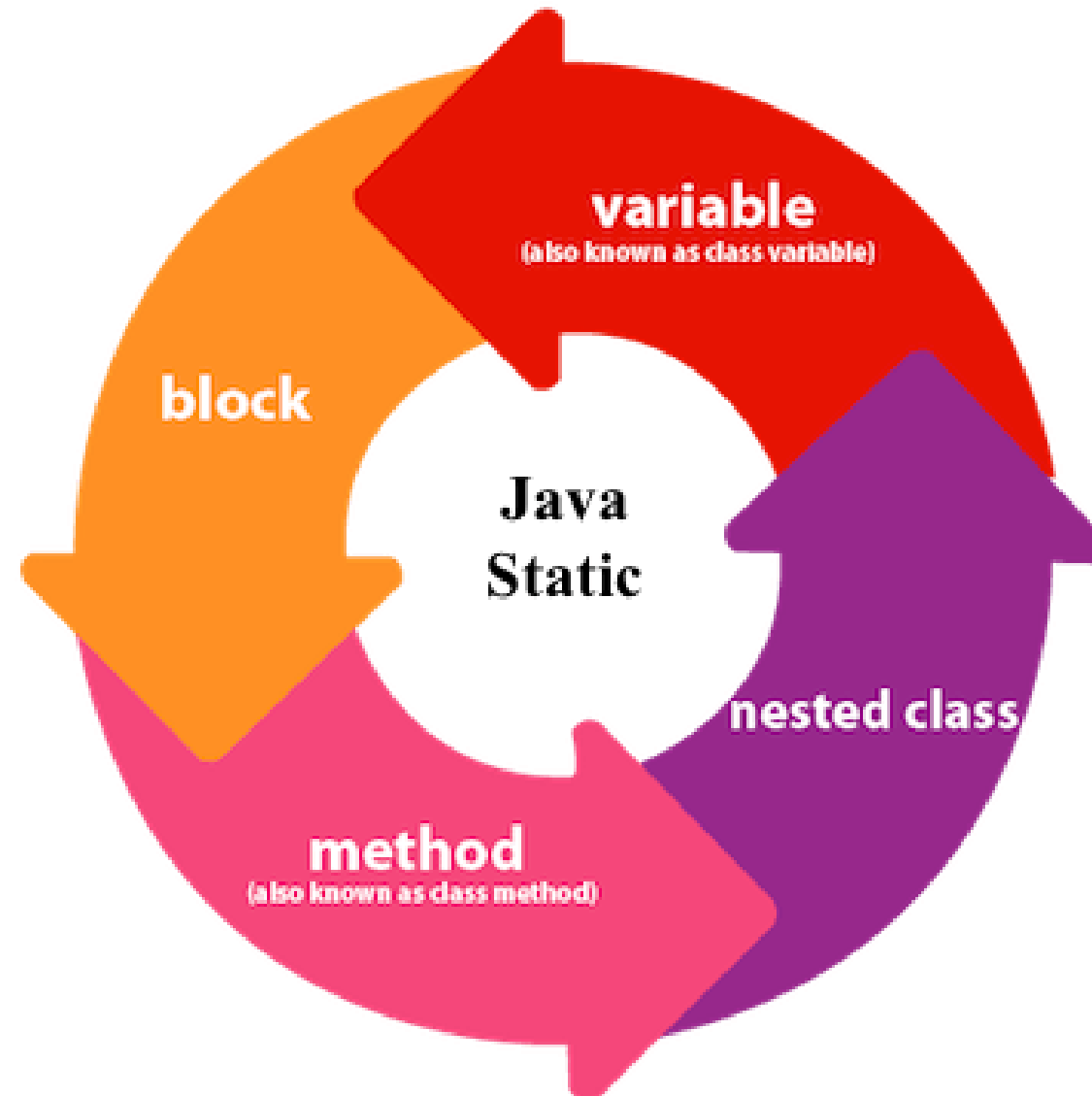
```
class Box {  
    double width, height, depth;  
    // Default constructor  
    Box() {  
        width = height = depth = 0;  
    }  
    // Constructor with one parameter  
    Box(double len) {  
        width = height = depth = len;  
    }  
    // Constructor with three parameters  
    Box(double w, double h, double d) {  
        width = w;  
        height = h;  
        depth = d;  
    }  
    double volume() {  
        return width * height * depth;  
    }  
}
```

Constructor Overloading

```
class Student {  
    String name;  
    int age;  
    // Constructor  
    Student(String n, int a) {  
        name = n;  
        age = a;  
    }  
    // Copy Constructor  
    Student(Student s) {  
        name = s.name;  
        age = s.age;  
    }  
    void displayInfo() {  
        System.out.println("Name: " + name + ", Age: " + age);  
    }  
}
```

Copy Constructor

- 1) Write your own java code by using three access modifiers(public, private and protected)**
- 2) Write a sample program for object reference variable by creating two reference variable for one object.**
- 3) Write a Java program to demonstrates how 'this' can be used to access instance variables.**



- **Static members in Java, consisting of static fields (variables) and static methods, are class-level components.**
- **Unlike instance members, which require an object of the class to be accessed, static members can be accessed directly using the class name, making them shared among all instances of the class.**
- **This characteristic makes them suitable for representing properties or behaviors that are common to all objects of that class.**

Key Features of Static Members:

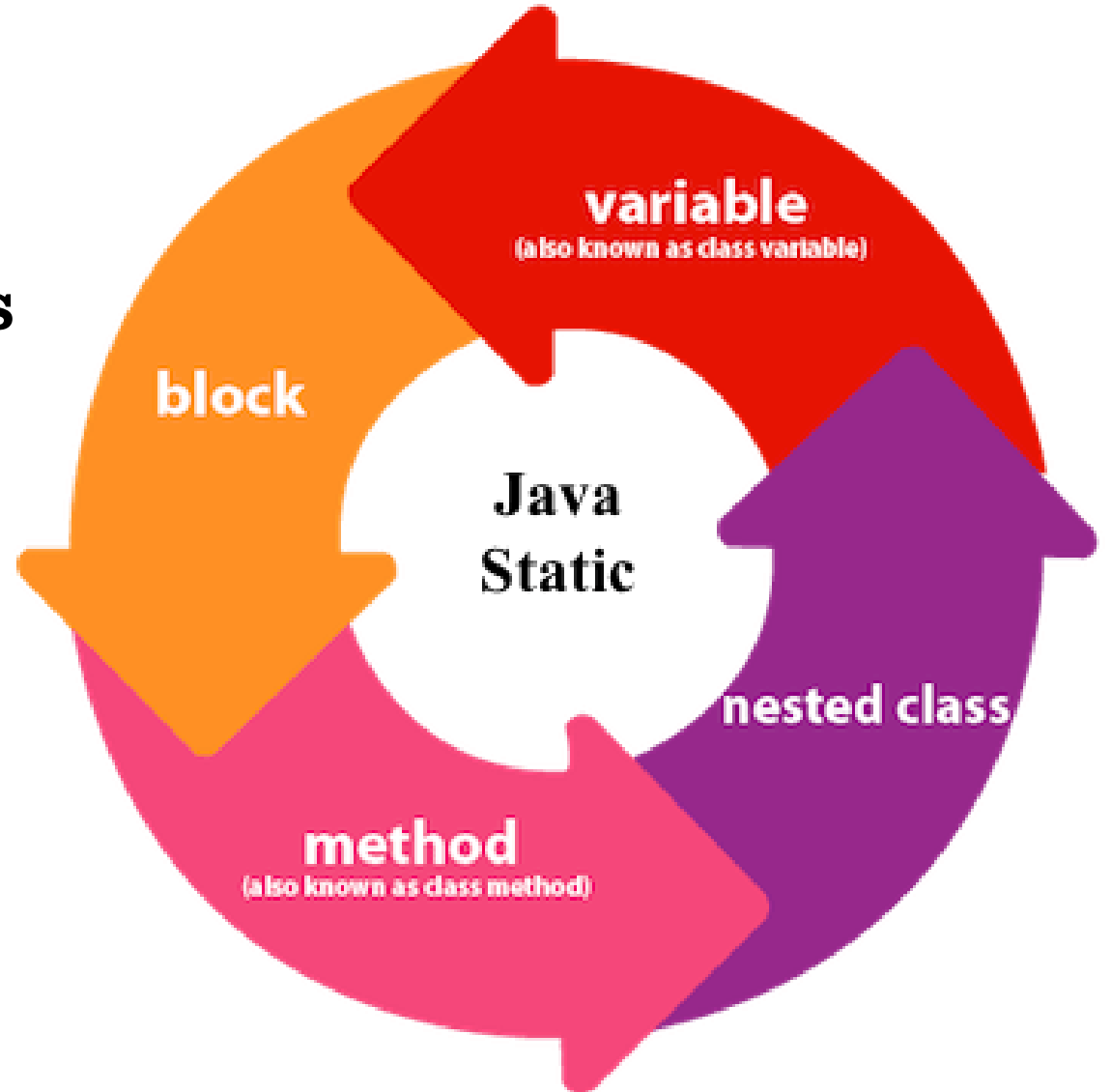
Shared among all instances: A static member is shared among all instances of the class, meaning all objects of the class have access to the same static variable or method.

Class level: Static members belong to the class itself, not to any specific instance.

Memory Efficiency: Since static members are shared, they are allocated space in memory only once, at the time of class loading, which makes them more memory efficient for shared data or methods.

Access: Static members are accessed using the class name, not the instance name, which allows them to be called even before any objects of the class are created.

- The static can be:
- Variable (also known as a class variable)
- Method (also known as a class method)
- Block
- Nested class



Imagine you have a notebook for each student in a class to write their name and roll number. Every student also needs to write down the name of their college. If the college name is the same for all students, it doesn't make sense for every student to write down the same college name in their own notebook, does it? This is similar to what happens when you don't use a static variable for the college name.

Without Static Variable:

Each student's notebook (object): Has their own name, roll number, and the college name written in it.

Result: If you have 500 students, the college name is written 500 times in 500 notebooks. This is a waste of space because the college name is the same for all.

With Static Variable:

College name: It's like having one big board in the classroom where you write the college name. Everyone can see it and knows it's their college.

Each student's notebook (object): Now only has their own name and roll number. They don't need to write the college name because it's already on the board.

Result: No matter how many students (objects) there are, the college name is stored only once on the board (static variable), saving a lot of space.

Without static:

```
class Student {  
    int rollno;  
    String name;  
    String college = "VITAP"; // Every student object has this, which is  
repetitive.  
}
```

With static:

```
class Student {  
    int rollno;  
    String name;  
    static String college = "VITAP"; // Stored once, all student objects  
share this info.  
}
```

Without Static Variable

```
class Book {
    String title;
    String author;
    String libraryName = "City Library"; // Non-static field

    public Book(String title, String author) {
        this.title = title;
        this.author = author;
    }
}

public class Main {
    public static void main(String[] args) {
        Book book = new Book("1984", "George Orwell");

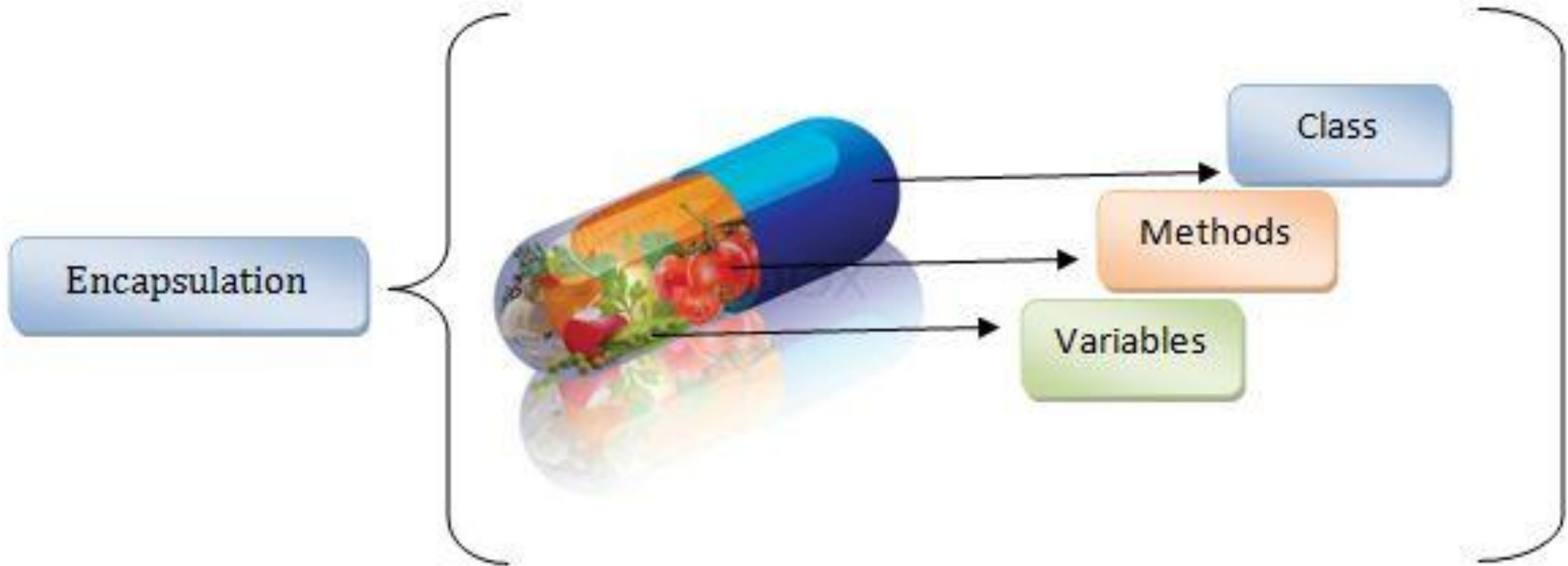
        // At this point, book.title is "1984", book.author is "George Orwell",
        // and book.libraryName is "City Library".
    } }
```

With Static Variable

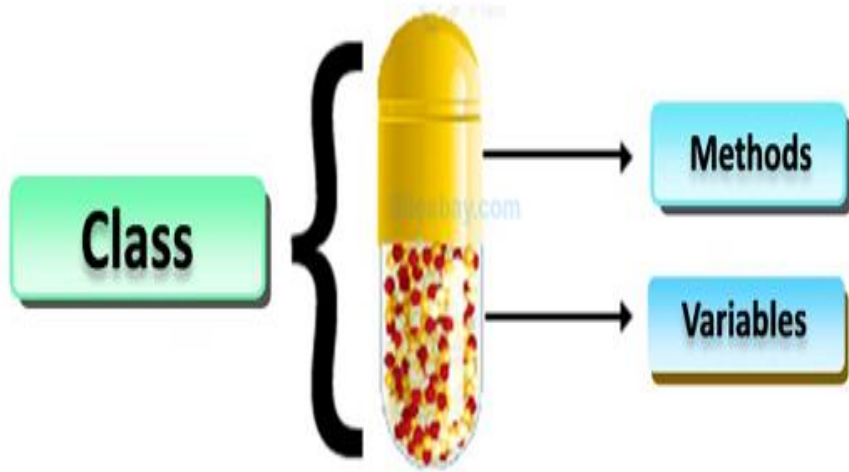
```
class Book {  
    String title;  
    String author;  
    static String libraryName = "City Library"; // Static field  
  
    public Book(String title, String author) {  
        this.title = title;  
        this.author = author;  
    }  
  
    public static void printLibraryName() {  
        System.out.println("Library Name: " + libraryName);  
    }  
}
```

By declaring libraryName as static, we ensure that it is stored only once in memory, even if we create hundreds or thousands of Book instances. This is a more memory-efficient way to handle common properties.

ENCAPSULATION



Encapsulation in Java



- Encapsulation is a fundamental concept in object-oriented programming (OOP) that refers to the bundling of data (attributes) and methods (functions) that operate on the data into a single unit, or class, and restricting access to some of the object's components. This is usually achieved by:
- Making the variables of a class private, so they cannot be accessed directly from outside the class.
- Providing public getter and setter methods to access and update the value of the private variables.

Encapsulation offers several benefits, including:

Control over data: Prevents unauthorized parties from tampering with sensitive data.

Increased security: Provides the developer with the flexibility to make variables read-only or write-only.

Flexibility and maintainability: Allows the developer to change one part of the code without affecting other parts.


```
public class Counter {  
    private int count; // Private variable, encapsulated  
    // Constructor to initialize count  
    public Counter() {  
        this.count = 0; // Initialize count with 0  
    }  
    // Public method to get the value of count  
    public int getCount() {  
        return count;  
    }  
    // Public method to increase count by 1  
    public void increment() {  
        count += 1;  
    }  
    // Public method to decrease count by 1  
    public void decrement() {  
        if (count > 0) {  
            count -= 1;  
        }  
    }  
}
```